

CodeGuardian: A Compiler-Based Static Security Analysis Engine

Detecting Injection & Control-Flow Vulnerabilities via Intermediate Code Taint Propagation

COURSE ALIGNMENT

Compiler Design Theory (BCSE307L) & CD Lab (BCSE307P)

TEAM

Mayur R Keni (24BCE0686)

Nitin Yajatkumar (24BCE0571)

OBJECTIVE

Building a complete, scratch-built compiler pipeline that converts Mini-C source code into a Control Flow Graph and runs custom data-flow equations to intercept untrusted data before runtime execution.

Zero-Day Exploits & Late-Stage Costs



Proliferation of Zero-Day Injection Flaws

High-severity vulnerabilities like SQL Injection (SQLi) and Remote Code Execution (RCE) remain primary threats because untrusted user inputs enter execution sinks unvalidated.



Late-Stage Detection Cost

Dynamic application security testing (DAST) identifies bugs only during execution or production, incurring high remediation overhead and incomplete code path coverage.

Opacity & Theoretical Disconnect



The "Black-Box" Opacity of Existing SAST Tools

Commercial static analysis tools report line-number alerts without exposing the underlying data-flow propagation paths, leaving developers unable to trace intermediate propagation through conditional blocks and loops.



Theoretical Disconnect

Traditional academic compiler projects stop at syntax translation, failing to leverage compiler data structures (AST, TAC, CFG) for proactive software security.

Front-End & Middle-End Engineering

1

Construct a Custom Mini-C Compiler Pipeline

Build a robust front-end and middle-end utilizing Flex and Bison to parse imperative constructs (variables, branching, loops, functions) into an Abstract Syntax Tree (AST).

2

Lower AST to Three-Address Code (TAC)

Implement Syntax-Directed Translation (SDT) to linearize high-level constructs into quadruples/triples.

Analysis & Visualization

3

Synthesize Control Flow Graphs (CFG)

Partition TAC sequences into basic blocks using leader analysis and construct explicit directed control-flow edges.

4

Formulate a Monotone Data-Flow Taint Engine

Implement an iterative fixpoint algorithm evaluating transfer functions across basic blocks to track source-to-sink variable propagation.

5

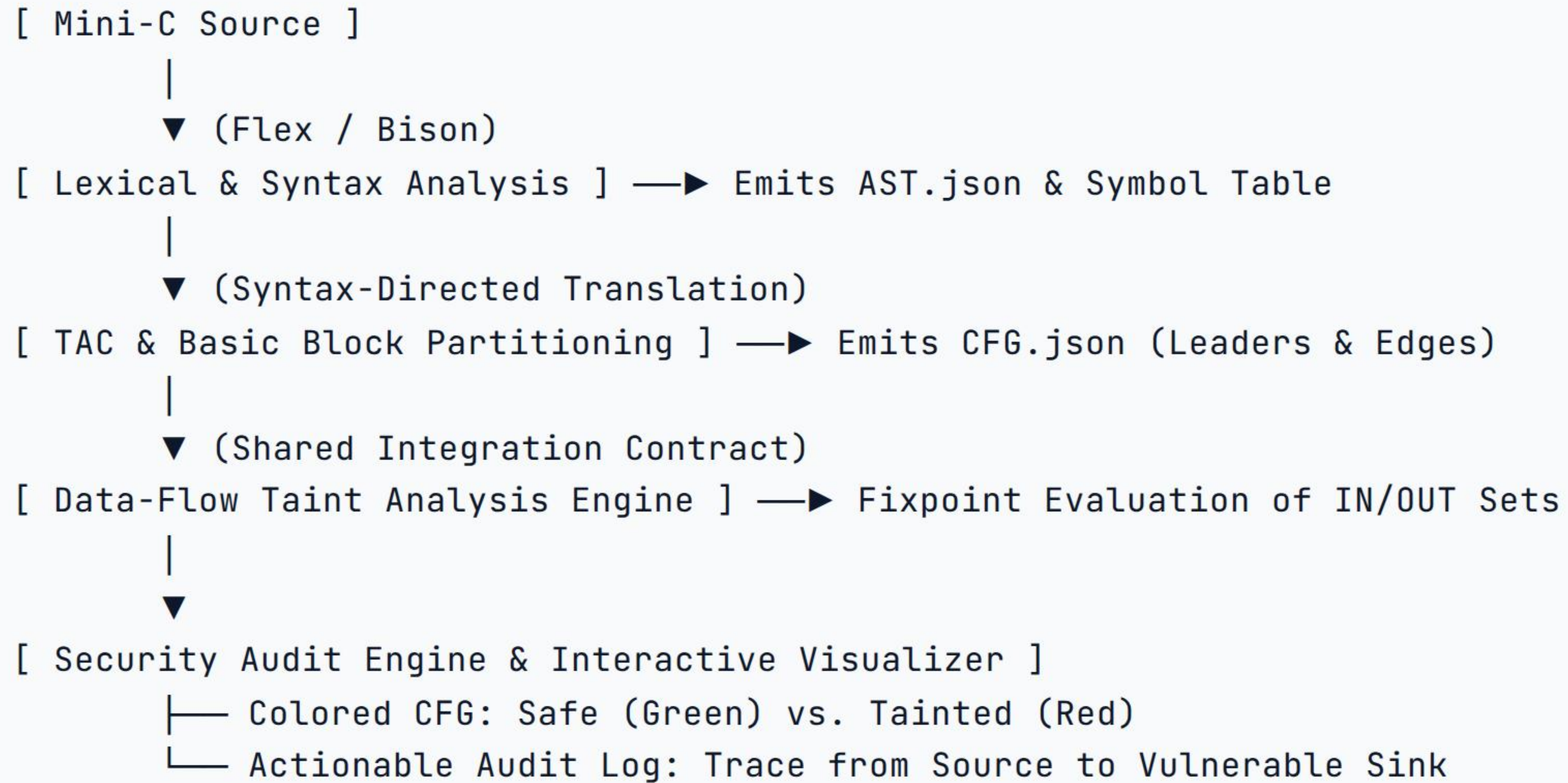
Deliver an Explanatory Security Dashboard

Render the generated CFG and highlight vulnerable paths in real time to expose compiler internals directly to the developer.

Current Methods vs. CodeGuardian

Tool / Methodology	Core Technique	Key Strengths	Identified Research & Usability Gap
Flawfinder / RATS	Lexical Regex Matching	Extremely fast; minimal resource overhead.	Context-unaware; triggers high false-positive rates; cannot track data flow across basic blocks.
Semgrep	Tree-sitter AST Pattern Matching	Expressive syntax-based rules; fast scanning.	Shallow data-flow tracking across complex loops and conditional branch merges.
GitHub CodeQL	Datalog Relational Queries	Enterprise-grade interprocedural analysis.	Steep learning curve; heavy compute requirements; closed/opaque intermediate representations.
CodeGuardian (Ours)	Scratch-Built Compiler Data-Flow Framework	Strict deterministic pipeline: AST → TAC → CFG → Iterative Fixpoint Analysis. Bridges the gap: transparent IR mapping with interactive visual verification.	

System Workflow



Contract Isolation: Front-end/middle-end synthesis outputs CFG.json, enabling asynchronous verification and zero pipeline bottlenecks between modules.

Advancing Static Analysis Theory



Zero-Dependency Core

The compiler pipeline is engineered from fundamental theory using Flex/Bison and custom C/C++ lowering logic rather than wrapping third-party parser generators.



Transparent IR Taint Propagation

Unlike commercial scanners operating on source code, CodeGuardian linearizes high-level control structures into TAC quadruples to track state across temporary registers.



Visual Data-Flow Verification

Unifies graph theory with an interactive UI; developers inspect basic block leaders, TAC lowering, and the exact set convergence of vulnerable control paths.



Direct Theoretical Mapping

Turns formal textbook equations—often kept purely theoretical—into an operational, industrial-style security verification engine.

Basic Block Identification

Employs canonical leader criteria:

- 1 Instruction 1 is a leader.
- 2 Any target of a conditional or unconditional jump is a leader.
- 3 Any instruction immediately following a jump is a leader.

Fixpoint Equations & Taint Semantics

ITERATIVE FIXPOINT DATA-FLOW EQUATIONS

$$IN [B] = \bigcup_{P \in Pred [B]} OUT [P]$$

$$OUT [B] = GEN [B] \cup (IN [B] - KILL [B])$$

Taint Semantics

- **GEN[B]**: Set of variables assigned from untrusted sources within block B.
- **KILL[B]**: Set of variables passed through sanitization routines within block B.
- ▲ **Vulnerability Trigger: A vulnerability is flagged if a sink instruction consumes operand v such that $v \in IN[B]$ and $v \notin KILL[B]$.**

Workload Division

MEMBER 1: COMPILER CORE

(Theory & Front-End)

- Modules 1 & 2: Flex & Bison CFG parsing
- Module 4: Symbol Table & AST lowering
- Module 4/5: TAC syntax-directed rules
- Module 5: Leader identification, basic blocks, and CFG graph generation

Deliverable:

Complete pipeline from raw Mini-C source to exportable CFG.json

MEMBER 2: SECURITY INTELLIGENCE

(Math Engine & Full-Stack)

- Modules 5 & 6: Data-flow equations & iterative fixpoint calculation
- Source/Sanitizer/Sink taint tracking
- Backend integration & API pipeline
- Interactive visualizer & audit report

Deliverable:

Complete analysis engine and visual dashboard from CFG.json

DELIVERABLES

Expected Outcomes

- ✓ **Functional Parser & TAC Engine**
Deterministic parsing of Mini-C control flows with immediate syntax validation and normalized intermediate representations.
- ✓ **Exportable Graph Intermediary (CFG.json)**
Fully structured representation containing node IDs, TAC instruction arrays, predecessor/successor maps, and jump conditions.
- ✓ **Auditing Dashboard**
Left pane: Mini-C code editor.
Center pane: Synchronized Three-Address Code listing.
Right pane: Interactive directed graph with visual node highlighting (Green = Safe, Red = Tainted).
Bottom pane: Vulnerability breakdown identifying source origin, propagation path, and unescaped sink target.

Thank You

Any Questions?